

CMTAT Confidential Audit

The ZAMA logo is displayed within a yellow rectangular box. The word "ZAMA" is written in a bold, black, sans-serif typeface, centered horizontally and vertically within the box.

July 9, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	6
Medium Severity	9
M-01 Rule Engine Not Applied to Mint and Burn Leaving Issuance and Redemption Unscreened	9
Low Severity	10
L-01 Sequential Total Supply Disclosures Leak Individual Mint and Burn Amounts	10
L-02 ERC7984TokenAttributeModule Seeds Attributes Via a Skippable Internal Initializer	11
Notes & Additional Information	12
N-01 Missing Docstrings	12
N-02 Incomplete Docstrings	12
N-03 Floating Pragma	13
N-04 Prefix Increment Operator ++i Can Save Gas in Loops	14
N-05 Misleading Documentation	14
Conclusion	16
Appendix	17
Issue Classification	17

Summary

Type	RWA Tokenization	Total Issues	8 (7 resolved)
Timeline	From 2026-06-12 To 2026-06-24	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	1 (1 resolved)
		Low Severity Issues	2 (2 resolved)
		Notes & Additional Information	5 (4 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the [CMTA/CMTAT-Confidential](#) repository at commit [463087c](#).

In scope were the following files:

```
contracts
├── deployment
│   ├── CMTATConfidential.sol
│   ├── CMTATConfidentialLite.sol
│   ├── CMTATConfidentialRuleEngine.sol
│   └── CMTATConfidentialWhitelist.sol
├── modules
│   ├── CMTATConfidentialVersionModule.sol
│   ├── ERC7984BalanceViewModule.sol
│   ├── ERC7984BurnModule.sol
│   ├── ERC7984EnforcementModule.sol
│   ├── ERC7984MintModule.sol
│   ├── ERC7984PublishTotalSupplyModule.sol
│   ├── ERC7984RuleEngineModule.sol
│   ├── ERC7984TokenAttributeModule.sol
│   └── ERC7984TotalSupplyViewModule.sol
└── CMTATConfidentialBase.sol
```

System Overview

CMTAT Confidential is a regulated security token whose balances and transfer amounts are encrypted. It is built for issuers that need the compliance controls of a permissioned security token (pause, address freezing, forced seizure, allowlisting, and rule-based transfer restrictions) while keeping per-account balances and per-transfer amounts confidential. Confidentiality is provided by Zama's Fully Homomorphic Encryption (FHE) stack: balances and total supply are stored as encrypted 64-bit integers (**euint64**), and transfer amounts are submitted as ciphertext handles accompanied by zero-knowledge proofs of knowledge. The contracts in scope primarily compose three external systems, the OpenZeppelin Confidential Contracts ERC-7984 implementation (token mechanics), the CMTAT framework (compliance modules), and the **@fhevm/solidity** library plus Zama protocol infrastructure (encrypted arithmetic, access control, and decryption), and add a layer of role-gated mint, burn, forced-operation, observer, and supply-visibility logic on top.

The code is organized around a single abstract base, **CMTATConfidentialBase**, which wires the ERC-7984 token together with the CMTAT compliance base and a set of local FHE modules, and centralizes transfer gating, role authorization hooks, and the eight ERC-7984 transfer overrides. Four concrete deployment variants extend this base:

- **CMTATConfidential** is the full variant. It adds a registered total-supply observer list whose members are automatically re-granted decryption access to the total supply after every mint and burn.
- **CMTATConfidentialLite** omits the total-supply observer list to reduce contract size and per-operation gas, while retaining one-shot public supply disclosure.
- **CMTATConfidentialRuleEngine** routes transfers through an external CMTA **RuleEngine** for policy checks.
- **CMTATConfidentialWhitelist** adds an on/off CMTAT allowlist that both the sender and the recipient must satisfy when enforcement is enabled.

Each local feature follows a consistent module pattern: a dedicated role constant, a modifier that calls a virtual authorization hook, and an optional validation hook, with the authorization hooks overridden in **CMTATConfidentialBase** to bind each capability to a specific role. Minting and burning accept encrypted amounts, validate the recipient or source against the active compliance modules, and update the encrypted total supply. Forced operations (**forcedTransfer** and **forcedBurn**) are seizure functions that require the source address

to be frozen and are designed to execute even when the contract is paused or deactivated. Standard transfers pass through `_canTransferGenericByModule`, which evaluates freeze and pause state (and, in the relevant variants, allowlist and rule-engine policy) before the encrypted transfer is performed.

Because all amounts are encrypted, the system carries several behaviors that follow directly from FHE arithmetic and shape its security posture. Transfers and burns that exceed the available encrypted balance do not revert; the operation saturates and moves zero, with no on-chain error. The `confidentialTransferAndCall` and `confidentialTransferFromAndCall` functions credit the receiver before invoking its callback, and the compensating reverse transfer used when the callback rejects can be defeated by a malicious or re-entrant receiver, so these functions are documented as safe only with trusted receiver contracts. Decryption access is governed by an access control list (ACL): once an account, observer, or the public is granted access to a ciphertext handle, that grant cannot be revoked, and amount-based policy rules cannot be evaluated on-chain because the value is hidden (rule-engine and view checks therefore always pass `0` as the amount). The `euint64` representation also bounds the maximum representable supply, and the constructor rejects token decimals above 18 for this reason.

Security Model and Trust Assumptions

The contracts implement a permissioned security token, so significant power is delegated to privileged roles by design. The roles below should be understood as trusted within their stated powers; the risk to evaluate is the consequence of a compromised or misbehaving role holder, not the existence of the power itself.

DEFAULT_ADMIN_ROLE: Manages the grant and revocation of every other role and can permanently deactivate the contract. In the full variant it also sets the cap on the number of total-supply observers. Because it controls role assignment, this role is effectively able to assume any other capability in the system and must be held by the most trusted party.

MINTER_ROLE and **BURNER_ROLE**: Mint encrypted amounts to arbitrary recipients and burn from arbitrary accounts, in each case subject only to the active compliance checks (deactivation, freeze, and depending on the deployment variant the configured transfer-policy layer such as allowlist gating or RuleEngine screening) on the affected account. These roles are

trusted not to inflate or destroy supply outside the issuer's intended policy, but are not trusted to bypass the deployment's compliance screening; consistent with upstream CMTAT, that screening is expected to apply to issuance and redemption as it does to transfers. Minting and burning are permitted while the contract is paused.

ENFORCER_ROLE and **FORCED_OPS_ROLE**: Together they form the seizure path.

ENFORCER_ROLE freezes and unfreezes addresses, and **FORCED_OPS_ROLE** executes forced transfers and forced burns against frozen addresses, including while the contract is paused or deactivated. The two powers are intentionally separated so that freezing an account and moving its funds can be held by different actors. A forced transfer requires a non-zero recipient, and both forced operations require the source to be frozen first, which creates an on-chain freeze-then-seize trail.

PAUSER_ROLE: Pauses and unpauses transfers. Pausing blocks confidential transfers but does not block mint, burn, or forced operations.

OBSERVER_ROLE and holder-assigned observers: **OBSERVER_ROLE** assigns a per-account "role observer" to any account, and each holder can independently assign their own observer. On every balance update both observers are granted ACL access to the account's new balance handle and to the transferred amount handle in `ERC7984BalanceViewModule._update`, which gives an observer transfer-level visibility (the ability to reconstruct individual transfer amounts off-chain), not merely a balance snapshot. Because ACL grants cannot be revoked, removing an observer only stops future grants; any handle already shared remains decryptable by that observer permanently. An observer holding ACL access to a handle can drive disclosure through the ERC-7984 flow (`requestDiscloseEncryptedAmount` and `discloseEncryptedAmount`), and even if that entrypoint is overridden or restricted, the observer can bypass it by calling `allowForDecryption` on the ACL contract directly. Handles are treated as secrets, and any party with access to the secret is treated equally.

SUPPLY_OBSERVER_ROLE and **SUPPLY_PUBLISHER_ROLE**: **SUPPLY_OBSERVER_ROLE** (full variants only) manages the list of addresses that automatically receive decryption access to the total supply after each mint and burn. **SUPPLY_PUBLISHER_ROLE** (all variants) can mark the current total supply handle as publicly decryptable, which is irreversible for that handle. Both roles deliberately reduce the confidentiality of total supply and are trusted to do so only as intended.

RULE_ENGINE_ROLE (RuleEngine variant): Sets and replaces the external `RuleEngine`, or disables it by setting the zero address. A malicious, broken, or codeless rule engine can block

transfers (by reverting or returning false), so this role and the configured engine are trusted to remain a functioning, intended policy contract.

ALLOWLIST_ROLE (Whitelist variant): Enables or disables allowlist enforcement and manages the allowlisted set. When enforcement is enabled, this role can censor transfers, mints, and burns by withholding allowlisting. Forced operations intentionally bypass the allowlist, requiring only that the source is frozen.

Metadata roles (**TOKEN_ATTRIBUTE_ROLE** , **EXTRA_INFORMATION_ROLE** , **DOCUMENT_ROLE**): Update the token name and symbol post-deployment, the token identifier, terms, and free-form information, and the attached ERC-1643 documents. These are treated as issuer-controlled mutable data rather than user-owned state.

External systems and integrators: Correctness and confidentiality rest on components outside this repository. The OpenZeppelin Confidential Contracts ERC-7984 implementation, the CMTAT compliance framework, and the CMTA **RuleEngine** are assumed to be present at the intended versions and to behave as specified. The Zama FHE coprocessor, ACL, verifier, and KMS infrastructure are assumed to validate encrypted inputs and proofs, enforce and record ACL permissions, and honor public-decryptability flags. Integrators carry two specific responsibilities: callers of **confidentialTransferAndCall** and **confidentialTransferFromAndCall** must use only trusted receiver contracts because the refund path is non-atomic and can be defeated by a re-entrant receiver, and operators of the observer mechanisms must keep observer lists small because each registered observer adds an **FHE.allow** call to every mint or burn, so an oversized list can make those operations fail on gas.

Deployment: The variants are intended to be deployed directly so that the constructor runs and initializes roles and configuration. The internal **initialize** function is not exposed as an external initializer, so deploying behind a proxy without running the constructor would leave the contract uninitialized.

Update: *The fix of the [L-01](#) issue introduced a new trust assumption: The **SUPPLY_PUBLISHER_ROLE** holder is trusted to always aggregate many supply-changing operations per disclosure. **SUPPLY_PUBLISHER_ROLE** will be restricted to a multisig/timelock rather than a high-frequency automated caller.*

Medium Severity

M-01 Rule Engine Not Applied to Mint and Burn Leaving Issuance and Redemption Unscreened

Standard CMTAT applies the rule engine at the `_update` chokepoint, so every balance change, including mint and burn, routes through `ruleEngine.transferred` and is screened. The confidential RuleEngine variant instead wires rule-engine enforcement at the `_beforeTransfer` hook, which is invoked only by the `confidentialTransfer` family of functions. As a result, `mint`, `burn`, `forcedTransfer`, and `forcedBurn` never consult the rule engine, diverging from the standard CMTAT enforcement point.

In the CMTA rule engine, `transferred` performs address-based screening and reverts when a party is not permitted. For example, `RuleWhitelist` rejects a non-whitelisted address, with sanctions and blacklist rules behaving similarly. Skipping this hook on issuance and redemption therefore removes genuine screening rather than only desynchronizing rule state. A mint to a non-whitelisted or sanctioned address can succeed even though a `confidentialTransfer` to the same address would revert, and a burn from such an address is likewise unscreened. For an issuer relying on the rule engine for KYC or sanctions enforcement, this allows tokens to be issued to or redeemed from addresses the rule engine would reject. Forced operations skipping the rule engine remain acceptable under the accepted forced-operations bypass assumption.

Consider applying the rule engine at the `_update` chokepoint as standard CMTAT does, so that mint and burn are covered, with appropriate handling for the zero address on each leg. Alternatively, if issuance and redemption are intentionally exempt from rule-engine screening, document that intent explicitly and ensure equivalent screening is enforced through the CMTAT controls.

Update: Resolved in [pull request #15](#) at commits [4e559b1](#) and [0b10f17](#). The CMTA team stated:

Forced operations (`forcedTransfer` / `forcedBurn`) intentionally continue to bypass the engine. Regression tests were added covering blocked/allowed mint and burn, the forced-ops bypass, and engine-disabled behaviour.

Low Severity

L-01 Sequential Total Supply Disclosures Leak Individual Mint and Burn Amounts

The `ERC7984PublishTotalSupplyModule` allows a holder of the `SUPPLY_PUBLISHER` role to call `publishTotalSupply`, which marks the current encrypted total supply handle as publicly decryptable. Each disclosure reveals only an aggregate, but repeated disclosures open a delta-inference channel. When the total supply is published as `V1`, then a mint or burn of confidential amount `X` occurs, and the total supply is published again as `V2`, any observer can compute the difference between the two values. If exactly one supply-changing operation occurred between the publications, that difference equals the individual mint or burn amount, which is otherwise confidential. If several occurred, the net change leaks. Both disclosures are permanent and global.

This weakens a confidentiality property of the token. For a regulated security token, mints and burns correspond to share issuance and redemption, whose amounts may be commercially sensitive. A `SUPPLY_PUBLISHER` acting in good faith, for example, publishing supply on a regular reporting cadence, can inadvertently reveal individual amounts that occur between publications, with no malicious actor required. The effect is tempered by the requirement that two voluntary publication actions occur, and the module documentation, while correct on per-handle irrevocability, is silent on this cross-publication channel.

Consider documenting the delta-inference risk in the `publishTotalSupply` NatSpec and operator guidance, requiring a minimum number of supply-changing operations or a minimum time or aggregation window between publications, and treating `publishTotalSupply` as a deliberate and governed disclosure action rather than a routine high-frequency one.

Update: Resolved in [pull request #15](#) at commit [de3f439](#). The CMTA team stated:

*Accepted as residual risk; mitigated by documentation. The cross-publication delta-inference channel (`| V2 - V1 |`) cannot be fully closed in code — a `SUPPLY_PUBLISHER_ROLE` holder can always disclose the aggregate. A `counter ≥ k` gate on `publishTotalSupply()` would give *k*-anonymity against external observers, but it only hides the exact split (not the per-window sum/magnitude) and — decisively — trades away the transparent, on-demand total supply, since the exact figure cannot be published right after a single operation. It was therefore deliberately not implemented (see [L-01 — In-code mitigation analysis](#)). (It does not need to defend*

against the trusted `MINTER_ROLE` / `BURNER_ROLE` operators, who already know their own operations.) We also prefer to leave the behavior as-is because CMTAT-Confidential is a library used by many issuers with different goals: some may deliberately want to publish the total supply after every mint/burn for maximal transparency, accepting the delta-inference leak as a conscious trade-off, and a hard-coded gate would deny them that. It is now documented in the `publishTotalSupply` NatSpec, the module docstring, the README operator guidance, together with the recommended operational mitigations: aggregate many supply-changing operations per disclosure, and restrict `SUPPLY_PUBLISHER_ROLE` to a multisig/timelock rather than a high-frequency automated caller.

Update: Resolved

L-02 ERC7984TokenAttributeModule Seeds Attributes Via a Skippable Internal_INITIALIZER

`ERC7984TokenAttributeModule` shadows the `name` and `symbol` storage of the non-upgradeable `ERC7984` base with its own `_name` and `_symbol` fields and overrides the `name()` / `symbol()` views to return them, enabling permissioned post-deployment updates. These fields are seeded by the internal function `_initTokenAttributes`, which an inheriting contract must remember to call from its constructor. `CMTATConfidentialBase` does so [in its constructor](#), passing the same `name_` / `symbol_` it forwards to the `ERC7984` constructor.

Because the project is deployed without a proxy, `ERC7984TokenAttributeModule` could instead declare a constructor, which Solidity constructor-chaining would force every inheriting contract to satisfy at compile time. As an internal initializer, `_initTokenAttributes` carries no such guarantee: a future deployment variant that forgets to call it, or a refactor that drops the call, compiles cleanly but leaves the module's `_name` / `_symbol` as empty strings. Since `name()` / `symbol()` resolve to the module's storage, the token would then report an empty name and symbol while the `ERC7984` base's own shadowed, never-read copies remain set correctly — a silent inconsistency with no compile-time signal.

Consider replacing `_initTokenAttributes` with a `constructor` on `ERC7984TokenAttributeModule`, so the seed is enforced by the compiler for every inheriting contract and cannot be silently omitted.

Update: Resolved in [pull request #15](#) at commit [8eb95f2](#).

Notes & Additional Information

N-01 Missing Docstrings

Throughout the codebase, multiple instances of missing docstrings were identified:

- The role constants are not documented: `OBSERVER_ROLE`, `BURNER_ROLE`, `FORCED_OPS_ROLE`, `MINTER_ROLE`, `SUPPLY_PUBLISHER_ROLE`, `RULE_ENGINE_ROLE`, `TOKEN_ATTRIBUTE_ROLE`, and `SUPPLY_OBSERVER_ROLE`.
- The `supportsInterface` function is not documented in `CMTATConfidentialBase.sol`, `CMTATConfidential.sol`, and `CMTATConfidentialWhitelist.sol`.
- The `version` function is not documented in `CMTATConfidentialBase.sol` or `CMTATConfidentialVersionModule.sol`.
- In `CMTATConfidential.sol`, the `confidentialTransfer*`, `decimals`, `name`, and `symbol` functions are not documented.

Consider thoroughly documenting all functions (and their parameters) that are part of any contract's public API. Functions implementing sensitive functionality, even if not public, should be clearly documented as well. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Resolved in [pull request #15](#) at commit [8d283f6](#).

N-02 Incomplete Docstrings

Throughout the codebase, multiple instances of incomplete docstrings were identified:

- The `from` and `to` parameters and the return value are not documented in the `canTransfer` function of `CMTATConfidentialBase.sol`, `CMTATConfidentialRuleEngine.sol`, and `CMTATConfidentialWhitelist.sol`.

- In `CMTATConfidentialRuleEngine.sol`, the `spender`, `from`, and `to` parameters and the return value are not documented in the `canTransferFrom` function.
- In `ERC7984RuleEngineModule.sol`, the `newRuleEngine` parameter is not documented in the `setRuleEngine` function.

Consider thoroughly documenting all functions/events (and their parameters or return values) that are part of a contract's public API. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Resolved in [pull request #15](#) at commit [20a87db](#).

N-03 Floating Pragma

A floating pragma allows a contract to be compiled with any version in the range, so it can be deployed with a compiler different from the one it was developed and tested against. This could potentially introduce unexpected behavior. Pinning the pragma ensures that the exact version used at deployment is consistent.

This issue pertains to final deployable contracts; abstract contracts and modules that are only inherited may retain a floating pragma. The deployable variants currently use the floating pragma `pragma solidity ^0.8.27;`:

- `CMTATConfidential.sol`
- `CMTATConfidentialLite.sol`
- `CMTATConfidentialRuleEngine.sol`
- `CMTATConfidentialWhitelist.sol`

Consider fixing the pragma in the deployable contracts.

Update: Not an issue. The CMTA team stated:

Won't fix (by design). CMTAT-Confidential is intended to be consumed as a library, not only as a fixed set of pre-compiled deployables. Pinning `pragma solidity 0.8.27;` would force every downstream integrator to compile with exactly that version; the floating `^0.8.27` is deliberate so library users can choose the `0.8.x` compiler appropriate for their own project. Deployers who require a

reproducible build should pin the compiler version in their own build configuration (this repository currently compiles and tests against 0.8.34).

N-04 Prefix Increment Operator `++i` Can Save Gas in Loops

The use of the post-increment operator `i++` in `ERC7984TotalSupplyViewModule.sol` could be more efficient.

Consider using the prefix increment operator `++i` instead of the post-increment operator `i++` when the return value of the expression is ignored, as this can save gas by skipping the storage of the pre-increment value.

Update: Resolved in [pull request #15](#) at commit [28090a8](#).

N-05 Misleading Documentation

Several NatSpec comments either misdescribe the actual call structure or omit warnings present on equivalent functions. These mismatches between documentation and implementation can mislead future readers and integrators about the contracts' behavior and trust assumptions.

- The operator-based `confidentialTransferFromAndCall` overloads omit the non-atomic refund warning that both `confidentialTransferAndCall` overloads carry, relying only on `@inheritdoc ERC7984`. The `confidentialTransferAndCall` warning states that the receiver is credited before its callback executes and that the best-effort refund on failure may silently fail after reentrancy, advising use only with trusted receiver contracts. Since both families share the same credited-before-callback pattern and best-effort refund semantics, the missing parity increases the likelihood that integrators treat `confidentialTransferFromAndCall` as an atomic pay-and-call primitive and invoke it with untrusted receivers, risking irreversible fund loss in callback-driven flows.
- `CMTATConfidentialBase._afterBurn` states that it delegates to `super` so that the full hook chain is preserved, but it calls `ERC7984BurnModule._afterBurn` directly. Both `_afterBurn` hooks are empty, so no hook chain is preserved.
- `CMTATConfidential` states that each body delegates to `super`, but the bodies call the corresponding `CMTATConfidentialBase` functions explicitly.

- `ERC7984TokenAttributeModule` refers to the `ERC7984(name_, symbol_, ...)` invocation as a `super` call, but it is a base-constructor invocation rather than a `super` call.

Consider adding the silent refund failure warning to the `confidentialTransferFromAndCall` overloads, or referencing the existing `confidentialTransferAndCall` warning, and stating explicitly that both entry point families share the same refund and receiver-trust assumptions; consider also correcting the comments that describe super delegation so they reflect the actual call structure.

Update: Resolved in [pull request #15](#) at commit [104218a](#).

Conclusion

CMTAT Confidential implements a regulated security token whose balances and transfer amounts remain encrypted, combining the OpenZeppelin Confidential Contracts ERC-7984 token with the CMTAT compliance framework and Zama's Fully Homomorphic Encryption infrastructure. OpenZeppelin audited the shared base contract, its four deployment variants, and the local modules that add role-gated minting, burning, forced operations, balance and supply observation, and total supply disclosure.

The in-scope contracts act primarily as a composition layer over established external systems, and the issues identified during the audit cluster at the boundary between the token's confidentiality goals and its compliance enforcement. The most consequential concerns the rule-engine variant: because rule-engine screening is wired into the confidential transfer path rather than the shared balance-update chokepoint that standard CMTAT uses, minting and burning bypass it entirely, so tokens can be issued to or redeemed from addresses the rule engine would otherwise reject for KYC or sanctions reasons. A separate confidentiality observation concerns public total supply disclosure: although each disclosure reveals only an aggregate, repeated disclosures around a small number of supply-changing operations allow any observer to infer individual mint or burn amounts by differencing, which can happen even with a publisher acting in good faith on a regular reporting cadence. A third, lower-impact observation concerns the post-deployment name and symbol mechanism, which is seeded by an internal initializer rather than a constructor and could leave a future deployment variant reporting an empty name and symbol with no compile-time signal.

These findings, together with the system's heavy reliance on privileged roles and on the upstream ERC-7984, CMTAT, and Zama components behaving as specified, share a common theme: the token's security depends as much on how confidentiality and compliance are wired together and operated as on the individual contracts. The behaviors that follow from encrypted arithmetic, including the silent saturation of over-limit transfers, the non-atomic refund path in the transfer-and-call functions, and the irrevocability of access control grants, reinforce this. Applying compliance screening at a single chokepoint so that issuance and redemption are covered, treating total supply disclosure as a deliberate and governed action rather than a routine one, and enforcing initialization at compile time would each strengthen that posture.

OpenZeppelin is grateful to the Zama team for their cooperation and responsiveness throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.